

5 STRINGS

In Python a string is a sequence of Unicode code points. These code points are converted into a sequence of bytes for efficient storage. This process is called character encoding. There are many encodings such as UTF-8, UTF-16, ASCII etc. and Python uses UTF-8 encoding by default.

5.1 The Story of Unicode

ASCII can only represent a limited number of characters at one time. It isn't very useful to represent any language that isn't based on a Latin character set. ASCII only defines 128 characters. So only 7 bits is normally used in ASCII which is not sufficient to support characters that required more bits to represent. At present every computer program must be scalable enough to take care various international characters and an application needs to be internationalized to display the same output or warning message in a variety of languages in English, French, Japanese, Hebrew, or Russian. Web content can be written in any of these languages and can also include a variety of emoji symbols. Python's string type uses the Unicode Standard for representing characters, which lets Python programs work with all these different possible characters.

However, UTF-8 which is an encoding standard for UCS-4 (Unicode) can represent almost any language. It does this by chaining multiple bytes together to represent one character (or glyph to be more correct). A character is represented on a screen or on paper by a set of graphical elements that's called a glyph. The glyph for an uppercase A, for example, is two diagonal strokes and a horizontal stroke, though the exact details will depend on the font being used. Most Python code doesn't need to worry about glyphs; figuring out the correct glyph to display is generally the job of a GUI toolkit or a terminal's font renderer.

5.1.1 Encoding

Unicode string is a sequence of code points, which are numbers from 0 through 0x10FFFF (1,114,111 decimal). This sequence of code points needs to be represented in memory as a set of code units, and code units are then mapped to 8-bit bytes. The rules for translating a Unicode string into a sequence of bytes are called a character encoding, or just an encoding.

The first encoding you might think of is using 32-bit integers as the code unit, and then using the CPU's representation of 32-bit integers. In this representation, the string "Python" might look like this:

P				y				t				h				o				n			
0x50	00	00	00	79	00	00	00	74	00	00	00	68	00	00	00	6f	00	00	00	6e	00	00	00
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23

This representation is straightforward but using it presents a number of problems. Such as:

- It's not portable; different processors order the bytes differently.
- It's very wasteful of space. In most texts, most of the code points are less than 127, or less than 255, so a lot of space is occupied by 0x00 bytes. The above string takes 24 bytes compared to the 6 bytes needed for an ASCII representation. Increased RAM usage doesn't matter too much (desktop computers have gigabytes of RAM, and strings aren't usually that large), but expanding our usage of disk and network bandwidth by a factor of 4 is intolerable.
- It's not compatible with existing C functions such as `strlen()`, so a new family of wide string functions would need to be used.

Therefore, this encoding isn't used very much, and people instead choose other encodings that are more efficient and convenient, such as UTF-8. UTF-8 is one of the most used encodings, and Python often defaults to using it. **UTF** stands for **"Unicode Transformation Format"**, and the '8' means that 8-bit values are used in the encoding. (There are also UTF-16 and UTF-32 encodings, but they are less frequently used than UTF-8.) UTF-8 uses the following rules:

- If the code point is < 128, it's represented by the corresponding byte value.
- If the code point is >= 128, it's turned into a sequence of two, three, or four bytes, where each byte of the sequence is between 128 and 255.

UTF-8 has several convenient properties:

- It can handle any Unicode code point.
- A Unicode string is turned into a sequence of bytes that contains embedded zero bytes only where they represent the null character (**U+0000**). This means that UTF-8 strings can be processed by C functions such as `strcpy()` and sent through protocols that can't handle zero bytes for anything other than end-of-string markers.
- A string of ASCII text is also valid UTF-8 text.
- UTF-8 is compact; most used characters can be represented with one or two bytes.
- If bytes are corrupted or lost, it's possible to determine the start of the next UTF-8-encoded code point and resynchronize. It's also unlikely that random 8-bit data will look like valid UTF-8.
- UTF-8 is a byte oriented encoding. The encoding specifies that each character is represented by a specific sequence of one or more bytes. This avoids the byte-ordering issues that can occur with integer and word oriented encodings, like UTF-16 and UTF-32, where the sequence of bytes varies depending on the hardware on which the string was encoded.

5.2 String Literals

A string literal is a sequence of characters enclosed in quotes that represents text data in a programming language. It's a fixed value that cannot be changed during the program's execution. String literals are used for various purposes, such as displaying messages, storing text data, and performing string

manipulations. Python offers several ways to create string literals, providing flexibility for different use cases.

5.2.1 Single line string literals

String literals in python are surrounded by either single quotation (‘ ’) marks, or double quotation marks (“ ”). In python single and double-quoted strings are the same and return the same type of object. This type of objects supports only single line string only, as shown below:

```
>>s = 'Hello' # a single quoted string literal
>>print(s)
Hello
>>type(s)
<class 'str'>
>>s = "Hello" # a double quoted string literal
>>print(s)
Hello
>>type(s)
<class 'str'>
```

Python supports triple quoted string literals and a line break is part of the string and using triple quotes string literals are known as a multiline string.

```
>>s = '''This is a multiline
string surrounded by single quotes'''
>>print(s)
```

```
This is a multiline
string surrounded by single quotes
```

```
>>s = """This is a multiline
string surrounded by double quotes"""
>>print(s)
```

```
This is a multiline
string surrounded by double quotes
```

5.2.2 Raw String

A raw string is a type of string that can also include one or many escape characters. A raw string is formed by prefixing an 'r' or 'R', a character following a backslash is included in the string without change, and all backslashes are left in the string. For example, the string literal `r"\n"` consists of two characters: a backslash and a lowercase 'n'.

```
>>> s = r'\n'
>>> print(s)
\n
```

String quotes can be escaped with a backslash, but the backslash remains in the string; for example, `r"\\"` is a valid string literal consisting of two characters: a backslash and a double quote.

```
>>> s = r"\\"
>>> print(s)
\"
```

Any raw string contains only one backslash, `r"\\"` is not a valid string literal (even a raw string cannot end in an odd number of backslashes). Specifically, a raw string cannot end in a single backslash (since the backslash would escape the following quote character). Note also that a single backslash followed by a newline is interpreted as those two characters as part of the string, not as a line continuation.

```
>>> s = r'\
... Hello'
>>> s
'\\nHello'
```

5.3 Escape Characters

Escape characters are used to solve the problem of using special characters inside a string declaration. It directs the interpreter to take suitable actions mapped to that character. We denote escape characters with a backslash (`\`) at the beginning. For example, to put a new line in the string we can add a linefeed (`\n`). Table 5.1, provides information about all the escape characters supported by Python.

Table 5.1 List of Escape Characters and their purpose

Escape Character	Meaning
<code>\'</code>	Represents a single quote
<code>\n</code>	Feeding a newline
<code>\r</code>	Carriage return
<code>\t</code>	Considers a horizontal tab space
<code>\b</code>	A added a backspace
<code>\f</code>	A form feed
<code>\ooo</code>	Representing string by octal value
<code>\xhh</code>	Representing a hex value
<code>\\</code>	Add a backslash
<code>\uxxxx</code>	Represents a 16-bit hex value
<code>\xxxxxxxx</code>	Represents a 32-bit hex value

To add a horizontal tab space between two words '`\t`' need to be inserted between two characters, as follows:

```
>>> print('space between a \t b') #a horizontal space between a, b
space between a      b
```

String containing a `\n` shifted the characters to the next line. In the given example, words after `'\n'` i.e., `'next line'` displayed to the next line.

```
>>print('After new line \n next line')# a newline feed
After new line
next line
```

A string containing a backslash (`\`) can be formed by inserting two backslashes. A string can also be formed using the octal Unicode values for each character. Illustration given below shows an example of forming string literal `'Python'` using octal values:

```
>>ord('P'),ord('y'),ord('t'),ord('h'),ord('o'),ord('n') #Unicode vaules
(80, 121, 116, 104, 111, 110)
>>oct(80), oct(121), oct(116), oct(104), oct(111), oct(110) # Unicode to Octal nums
('0o120', '0o171', '0o164', '0o150', '0o157', '0o156')
>>print('\120\171\164\150\157\156') #Froming a string using octal characters
Python
```

5.4 Modifying String Object

In a nutshell, you can't. Strings are one of the immutable types in Python, meaning not able to be changed. Trying to modify a part of string through indexing (discussed later in this chapter) will generate a `TypeError`.

```
>>str1 = 'Python'
>>str1[0] = 'p'
Traceback (most recent call last):
  File "<pyshell#16>", line 1, in <module>
    str1[0] = 'p'
TypeError: 'str' object does not support item assignment
```

In general, python offers many ways to modify a string. You can usually easily accomplish what you want by generating a copy of the original string that has the desired change in place.

```
>>> str1 = 'Python'
>>> str1 = 'p' + str1[1:]
>>> str1
'python'
```

In the above code at `'p'+str1[1:]`, `str1` has been modified by slicing and concatenation operation (+). Using slicing (`str1[1:]`), we can access any subpart of a string, by proving its boundary. There is also a built-in string method to accomplish this:

```
>>> str1 = 'Python'
>>> str1.replace('P', 'p')
'python'
```

You have already seen the use of plus (+) and multiplication (*) operators with numeric operands in Chapter 4. These two operators can be applied to strings as well. The plus '+' operator concatenates two strings.

```
>>>S1 = 'Python'
>>>S2 = ' Language'
>>>S1 + S2
'Python Language'
>>>'This is String in ' + S1 + S2
'This is String in Python Language'
```

Two or more strings can also concatenate without using '+' operator, as follows:

```
>>>s = 'a' 'b' 'c'
>>>s
'abc'
```

The * operator creates multiple copies of a string. If **S** is a string and **n** is an integer, either of the following expressions returns a string consisting of **n** concatenated copies of **S**:

```
>>> 'abc'*3
'abcbcbcb'
>>> 3*'abc'
'abcbcbcb'
```

The multiplier operand **n** must be an integer and if the multiplier is zero or any negative number then result will be an empty string:

```
>>> s = 'Hello'
>>> s * 0
''
>>> s * -2
''
```

5.4 Comparing Strings

Many operators are used in performing the comparison of strings, such as equality operator (==), and comparison operators like (<, >, <=, >=, !=). Strings are compared, by comparing characters of each string at same position. It will check the character with a lower Unicode and is considered a smaller value character or larger value character, or equal value character. In Python, strings are compared by the comparing of ordinal value of characters. Python uses objects with the same values in memory which makes comparing objects faster. Some basic comparison operators are equal to (==) and 'is' operators.

Next some examples are illustrated to understand the string comparison and while comparing following points need to be consider:

- Comparisons are case sensitive, meaning 'G' is not the same as 'g'.

- Each character in a string has an ordinal value, which is what operators look out for, and not the actual character. For example, 'G' and 'g' have an ordinal value 71 and 103 respectively. So, when compared 'g' becomes greater than 'G'.

The equal comparison '==' operator returns **True**, if two strings are equal, else return **False**:

```
>>>'Python' == 'Python'
True
>>>'Python' == 'python'
False
```

The Boolean value **True** returned because both the operands of '==' are same and in the second comparison, False, return, because 'P' and 'p' are not same. Not Equal (!=) operator is opposite of equal (==) operator and checks for non-equal string objects:

Note: Remember that using = would make the interpreter assume you want to assign one value to another, make sure you are using == operator for comparison.

The less than (<), operator compares the string based on their ordinal values of each character. The less than (<) and other comparison operators requires two operands in the form **str1<str2**.

```
>>> string1 = "Hello"
>>> string2 = "hello"
>>> if string1 < string2:
...     print("string1 is less than string2")
... else:
...     print("string2 is less than string1")
...
string1 is less than string2
```

Consider the outcome of the code snippet, which states that **string1** is less than **string2**. This comparison is performed on the ordinal value of each character. To make this comparison more understandable further two string objects are taken by taking different case in the last character only: See the ordinal values of corresponding characters of each string objects:

```
>>> string1 = "Hello"
>>> string2 = "Hell0"
>>> for i in range(0, len(string1)):
...     print(f"{string1[i]} = {ord(string1[i])}, {string2[i]} = {ord(string2[i])}")
...
H = 72, H = 72
e = 101, e = 101
l = 108, l = 108
l = 108, l = 108
o = 111, 0 = 79
```

From the output, we can conclude that, all the characters are same except the last one i.e., 'o' vs. '0'. The ordinal value of 'o' is 111, greater than uppercase 0, whose ordinal value is 79. It means,

string2 is less than **string1**. Like operator `<`, Python also allows other comparison operators for string comparison based on ordinal values.

String objects can also compare with identity operator **is** and **is not**. As compared to the identity operator, the comparison operator `==` compares the values of string objects and checks for value equality. Whereas **is** operator checks whether both the string objects (operands) refer to the same memory reference or not. Consider the examples:

```
>>> str1 = 'Hello'
>>> str2 = 'Hello'
>>> str1 is str2
True
>>> str1 == str2
True
```

Let's test the identity of two string objects with same value and modifying any of them and check their memory reference with `id` for comparison:

```
>>> str1 = 'Python'
>>> str2 = str1
>>> id(str1), id(str2)
(140718039720784, 140718039720784)
>>> str1 is str2
True
```

Here `str1`, `str2` both referring to the same memory and comparing with operator `is` returns `True`. Now, modify the `str1` and concatenate "3.10" with `str1` and assigned "Python 3.10" to `str3` and compare them:

```
>>> str1 = str1 + " 3.10"
>>> str3 = "Python 3.10"
>>> str1 is str3
False
>>> id(str1), id(str2)
(1859705773744, 140718039720784)
>>> str1 == str3
True
```

Here, **str1** and **str3** both have same value as 'Python 3.10', but they are referring different memory address. So `is` operator returns `False`, but comparison operator `==` returns **True**, because operator `==`, compares values of **str1** and **str3**.

5.5 String Indexing and Slicing

5.5.1 Accessing string through index

We can consider a string object as an ordered set of items and all these items can be accessed using an index. Indexing is a process of accessing individual items using a number known as location or index. In Python, strings are ordered sequences of character data, and support indexing. Individual characters in a string can be accessed by specifying the string name followed by a number in square brackets (`[]`).

Strings are zero (0) index-based, which means the first character can be accessed using the number 0 by passing it inside the square brackets (`[]`). For a string object 'python', indexing for each character are shown below and these characters can be accessed using the index:

0	1	2	3	4	5
p	y	t	h	o	n

```
>>> s = 'python'
>>> s[0], s[1], s[2], s[3], s[4], s[5]
('p', 'y', 't', 'h', 'o', 'n')
```

The index of the last character is one less than the length of the string. The `len(s)`, will return the number of characters and subtracting 1 from it will get the last valid index, as follows:

```
>>> s[len(s)-1]
'n'
```

String indices can also be specified with negative numbers, in which case indexing occurs from the end of the string and the last character can be accessed using index -1, and to refer the second-to-last character by -2 and so on.

```
>>> s[-6], s[-5], s[-4], s[-3], s[-2], s[-1]
('p', 'y', 't', 'h', 'o', 'n')
```

Attempting an index beyond the range as positive or negative results an error and the python interpreter will raise an `IndexError`. For example, the string 'Python', -6 is the lowest negative index and 5 will be the highest positive index.

5.5.2 Getting Substring through slicing

Slicing is an indexing technique used to extract a substring from a string. If `s` is a string, an expression in the form `s[start : end]`, returns the portion of `s` starting with position `start`, and up to but not including position `end`.

```
>>> str1 = 'This is python language'
>>> str1[4:10]
' is py'
```

The above indexing returns the substring ' is py' (from space at the index 4 to index 9, i.e. y). The index of character `t` is 10, can get using `str1[10]`. It is obvious that, the indexing didn't include the character at index 10, because the `slicing [4:10]` returns `10-4=6` characters, not including the end index. If we omit the start index python interpreter by default, consider zero (0) as a start index. It means `str1[0:end]` is equivalent to the slicing `str1[:end]`.

```
>>> str1[:10]
'This is py'
```

Like this, if we omit the `end` index it will consider the length of the string as last index. It means `str1[0:len(str1)]` is equivalent to `str1[0:]`. For a given index `n` ($0 \leq n \leq \text{len}(n)$) the expression `str1[:n]+str1[n:]` will return the string object as same as `str1`. The first operand

`(str1[:n])` of the expression returns a substring from index `0` to `(n-1)` and the second operand `(str1[n:])` will pick the characters from index `n` to `len(info)`.

```
>>> str1[:5] + str1[5:]
'This is python language'
```

For this example, the value of `n` is considered as `5`. So, the string `str1` is sliced at `'This '` and `'is python language'`. Later the plus `+` operator performs the concatenation of these two sliced strings. Omitting both the index will return the same string, not as a copy, it's a reference to the original string:

```
>>> str2 = str1[:]
>>> id(str1), id(str2)
(1348591302928, 1348591302928)
>>> str2
'This is python language'
```

During slicing, if the start index is equal or greater than the end index then interpreter returns an empty string. Negative indices can be used with slicing as well. `-1` refers to the last character, `-2` the second-to-last, and so on, just like indexing.

```
>>> str1[3:3]
''
>>> str1[10:3]
''
>>> str1[-8:]
'language'
```

While slicing a string, a third value as `step` part of the index `str[start : end : step]`, representing a step count or a stride value can be used to get a substring. To add the step, count an extra colon `:` is added after the stop index. This step value represents the number of character(s) that need to be jumped between two consecutive characters of a string. See the examples given below:

```
>>> str1[::2]
'Ti spto agae'
```

Negative step value also allowed, in which case Python steps backward through the string. In this case, the starting index should be greater than the ending index. See the examples given below:

```
>>> str1 = 'Python'
>>> str1[::-1]
'nohtyP'
```

Using negative stride value means the sliced string will be a reverse one, which means from the last character of the string object. There is another use of using a negative stride value and specifically used to reverse a given string. A string can be reversed by using any of these three techniques:

- Considering the last index as the start index with -1 as the stride value `str1[5::-1]`
- By providing only the stride value as -1 while skipping other indexes `str1[::-1]`
- By providing -1 as the start index and stride value while skipping the end index `str1[-1::-1]`

5.6 Built-in String Functions

Python supports many built-in type functions that can be used with a string object. This sections highlights all of them with a description and a sample illustration.

Function name	Description
<code>chr(65)</code>	Returns the Unicode character of the given number.
<pre>>>> chr(65) 'A'</pre>	
<code>ord()</code>	Returns the ordinal value of the given character.
<pre>>>>ord('A') 65</pre>	
<code>len(str)</code>	Returns the length of the string
<pre>>>>len('Hello') 5</pre>	
<code>str('str')</code>	Returns a string object
<pre>>>> str1 = str('Hello') >>> str1 'Hello'</pre>	

5.7 String Methods

Methods are similar to functions. A method is a specialized type of callable procedure that is tightly associated with an object. Like a function, a method is called to perform a distinct task, but it is invoked on a specific object and has knowledge of its target object during execution. The syntax for invoking a method on an object is as follows:

`object.method_name(argument(s))`

In the above syntax, **object** represents any string object and **method_name()** will be any of the method discussed in this section. Few of them may require single or more number of arguments or may not and this list of methods further categorized based of their outcome.

5.7.1 Case Conversion

Methods related to case conversion such as upper to lower or title case or swapping the letter cases are categorized into this group.

Method	Description
<code>str.capitalize()</code>	Converts the first character to uppercase
<pre>>>'python programming'.capitalize() 'Python programming'</pre>	
<code>str.lower()</code>	Converts the string to lowercase letter
<pre>>>'PYTHON'.lower() 'python'</pre>	
<code>str.upper()</code>	Converts the string to uppercase letter
<pre>>>'python'.upper() 'PYTHON'</pre>	
<code>str.swapcase()</code>	Interchange the uppercase to lowercase or vice versa
<pre>>>'Python Programming'.swapcase() 'pYTHON pROGRAMMING'</pre>	
<code>str.title()</code>	Covert the first character of each word to uppercase
<pre>>>'python programming'.title() 'Python Programming'</pre>	

5.7.2 Searching within a String

Python provides various string methods for searching a substring within a given string. Each method in this group supports an optional **start** and **end** arguments. These are interpreted as string slicing and the action of the method is restricted to the portion of the target string starting at position **start** and proceeding up to but not including character position at **end**. If **start** is specified but **end** is not, these methods apply to the portion of the target string from start to the end of the string.

Method	Description
<code>count(sub_string)</code>	Returns an integer specifying the number of occurrences of the given sub_string in a given string. Returns zero if sub_string is not found
<pre>>>'aab bba aab aab ba rta'.count('aa') 3 >>'aab bba aab aab ba rta'.count('bc')</pre>	

0

count(sub_string, start, end) – start and end both are optional and specifying the start and end index of sub_string within a given string. If end index is given then start index become compulsory.

```
>>'aab bba aab aab ba rta'.count('aa',0,10) # with start and end index
```

2

```
>>'aab bba aab aab ba rta'.count('aa',5) # with start index
```

2

startswith(suffix, star, end) Returns True, if the given string starts with **suffix**, else return False. Other two parameters are optional and specifies the search area.

```
>>'Python'.startswith('Py')# Searching in complete string
```

True

```
>>'Hello Python'.startswith('Py',6)# searching within 'Python'
```

True

```
>>'Hello Python'.startswith('Py',0,6) # searching within 'Hello '
```

False

endswith(suffix, star, end) Return True, if the given string ends with **suffix**, else return False. Other two parameters are optional and specifies the search area

```
>>'Python'.endswith('on') #Searching in complete string
```

True

```
>>'Hello Python'.endswith('Py',6) #Searching within Python
```

False

```
>>'Hello Python'.endswith('thon',0,6) #searching within 'Hello '
```

False

find(sub_string, star, end) Returns the start index of the **sub_string**, if it is present in the string. On multiple occurrences returns the lowest index. On absence of optional parameters complete string is considered.

```
>>'sun sun gun gun'.find('sun') # return 0 as lowest index
```

0

```
>>'sun sun gun gun'.find('un',4,10) # searching within 'sun gun gun'
```

5

```
>>'sun sun gun gun'.find('abc',4,10) # return -1 if not found
```

-1

rfind(sub_string, star, end) Returns the index of the **sub_string**, if it is present in the given string. On multiple occurrences returns the highest index.

```
>>'sun sun gun gun'.rfind('sun') # returns 4 as highest index
4
```

<code>index(sub_string, start, end)</code>	Search the sub_string within a given string and returns the lowest index, same as find() .
--	--

```
>>'sun sun gun gun'.index('sun') # return 0 as lowest index
0
```

<code>rindex(sub_string, start, end)</code>	Search the sub_string within a given string and returns the highest index, same as rfind() .
---	--

```
>>'sun sun gun gun'.rindex('sun') # returns 4 as highest index
4
```

The only difference between `index()` and `rindex()` with `find()` and `rfind()` is, through an exception as **ValueError** instead of -1, if the given sub_string is not found in the string, as follows:

```
>>'sun sun gun gun'.index('abc')
Traceback (most recent call last):
  File "<pyshell#17>", line 1, in <module>
    'sun sun gun gun'.index('abc')
ValueError: substring not found
```

Out of `find()` and `index()` method following points need to keep in mind when to prefer one over another:

- If you are sure about the presence of a substring in the given string, you can use either of the methods.
- If you are not sure about the presence of substring use `find()`.
- In case, you are using the `index()` method to find the index of the substring, use exception handling to avoid runtime crashes. The exception mechanism is discussed later.

5.7.2 Matching pattern within a String

This set of methods checks whether a given strings formed with a specific type of characters or not, such as all alphabetic characters, all digits, all upper or lowercase characters or not. Like this, Python also provide many constants defined in a module namely **string**, can be used using **import** statement.

Method	Description
<code>isalnum()</code>	Returns True if the string contains characters from A-Z or a-z or 0-9.

```
>>'Alp45'.isalnum()# characters from A-Z, a-z, 0-9
True'
```

```
>>'abgr'.isalnum()# characters from a-z
True
>>'123'.isalnum()# characters from 0-9
True
>>'are 123'.isalnum()# white_space is included
False
```

isalpha() – Returns True if the given string contains only alphabets.

```
>>'Alp45'.isalpha()#contains numeric values
False
```

isdecimal() – Return True if all characters in the string are decimal characters of base 10. Return False for subscript and superscript numbers.

```
>>> '123'.isdecimal()
True
>>> two = '\u2082'
>>> "H"+two+"O"
'H2O'
>>> two.isdecimal()
False
```

isdigit() – Returns True if the characters are from 0 – 9 only, irrespective of subscript or superscript digits.

```
>>> '2.45'.isdigit()
False
>>> two.isdigit()
True
```

isnumeric() – Used to determine whether a string consists entirely of numeric characters. It covers basic decimal digits, subscript and superscript digits, character with Unicode numeric value and currency numerators

```
>>> '123abc'.isnumeric()
False
>>> text = "\u0661\u0662\u0663" # Arabic-Indic digits
>>> text.isnumeric()
True
```

str.isidentifier() – Returns True if **str**, is a valid python identifier name, means satisfies naming

convention listed in Chapter 3.

```
>>> '_abc23'.isidentifier()
True
>>> '23'.isidentifier()
False
>>> 'else'.isidentifier()
True
>>> '@name'.isidentifier()
False
```

islower() Returns true if, the given string contains all the lowercase characters.

isupper() Returns true if, the given string contains all the uppercase characters.

isprintable() Returns true if the given string, contains all the printable characters.

A printable character is generally considered to be any character that can be displayed on the screen or printed on a physical device. This includes: alphanumeric characters (a-z, A-Z, 0-9), punctuation characters (!, @, #, \$, %, etc.), space character. Nonprintable characters are those characters defined in the Unicode character database as “Other” or “Separator”, excepting the ASCII space (0x20) which is considered printable.

```
>>> '\t'.isprintable()
False
>>> ' '.isprintable()
True
>>> '\n'.isprintable()
False
>>> 'aA01@'.isprintable()
True
```

isspace() - Return True if there are only whitespace characters in the string and there is at least one character, False otherwise.

istitle() - Return True if the string is a titlecased string and there is at least one character, for example uppercase characters may only follow uncased characters and lowercase characters only cased ones. Return False otherwise.

5.7.2 Formatting an output String

This set of methods used to manipulate the whitespace surrounding the text within the string. They are particularly useful for formatting output or creating aligned columns of text.

center(width, fill) - Returns a string containing the given string as centred of the given width and filled with default ASCII space character, else padded with **fill** character. Returns the given string, if the width

is same as the length of the given string.

```
>>'abc'.center(10) # filled with ASCII space
'  abc  '
>>'abcde'.center(3) # returns the same string
'abcde'
>>'abc'.center(10,'*') # center padded with '*'
'****abc*****'
```

expandtabs(tabsiz) – Return a copy of the string where all tab characters are replaced by one or more spaces, depending on the current column and the given tab size. It has an optional keyword argument **tabsiz** with default value 8.

```
>>>s = 'a\tab\tabc\t'.expandtabs()#Create a string of length 24
>>>s
'a      ab      abc      '
>>>len(s)
24
```

The **expandtabs()** method replaces tab characters (**\t**) in a string with spaces. It considers the tab positions to occur every **tabsiz** characters (default is 8, meaning tab positions are at columns 0, 8, 16, and so on). To expand the string, it works as follows:

- The current column position is set to zero.
- The string is examined character by character.
- If the character is a tab (**\t**), one or more space characters are inserted into the resulting string until the current column position reaches the next tab position. (The tab character itself is not copied.)

```
>>>s = 'a\n\r\tab'.expandtabs()
>>>len(s)
13
>>>s
'a\n\r      ab'
```

If the character is a newline (**\n**) or return (**\r**), it is copied and the current column is reset to zero. Any other character is copied unchanged and the current column is incremented by one regardless of how the character is represented when printed.

```
>>>s = '01\t012\t0123\t01234'.expandtabs(4)
>>>len(s)
21
>>>s
'01  012 0123   01234'
```

In the above example, **tabsize** is set to 4 and till encounters a `'\t'`, considering the characters `'01'` and next two spaces are considered, because tab position is 4. After that, out of 4 spaces 3 are taken by `'012'` so only one space is added and before the next `'\t'` already 4 or more characters are there and complete **tabsize** 4 is added.

join(iterable) – Return a string which is the concatenation of the strings inside an iterable object.

```
>>>' '.join(['Hello','Python'])#given string is space
'Hello Python'
>>>''.join(['1','2','3'])#no space so concatenate to a number
'123'
```

ljust(width, fillchar) – Return a string left justified and padded with the **fillchar**. Default is ASCII space. Original string will return if width is less than or equal to the length of given string.

```
>>>'abc'.ljust(5)#left aligned with space
'abc  '
>>>'abc'.ljust(5, '*')#left aligned with *
'abc**'
```

rstrip(char) – Return a copy of the string after removing the leading characters. If no **char** passed then leading spaces are removed, else all the occurrences of given characters are removed.

```
>>>'  spacious  '.rstrip()#leading spaces removed
'spacious  '
>>>'www.example.com'.rstrip('cmowz.')#removed all w with first period(.)
'example.com'
```

5.8 String Module

String module provides many constants and can be used to verify string contains all the alphabets, digits, punctuations, and whitespace etc. Table 5.2 listed all such constants with their purpose.

Table 5.2 Various constants available within string module

String Constants	Description
ascii_letters	Returns a string containing all ASCII lower and uppercase
ascii_lowercase	Returns a string containing all ASCII lowercase characters
ascii_uppercase	Returns a string containing all ASCII uppercase characters
digits	Returns a string containing all the digits from 0-9

hexdigits	Returns a string containing all the hex digits 0-9 and a-f and A-F
octdigits	Returns a string containing all the octal digits 0-7
punctuation	String of ASCII characters which are considered punctuation characters
printable	String of ASCII characters which are considered printable by combining <i>digits</i> , <i>ascii_letters</i> , <i>punctuation</i> and <i>whitespace</i>
whitespace	A string containing all ASCII characters that are considered whitespace. This includes the characters space, tab, linefeed, return, form feed, and vertical tab.

The `import string` statement facilitate us to use any of these constants, as follows:

```
>>import string
>>string.ascii_letters
'abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ'
>>string.digits
'0123456789'
>>string.printable
'0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ! " #$ % & \
' ( ) * + , - . / : ; < = > ? @ [ \ ] ^ _ ` { | } ~ \t \n \r \x0b \x0c '
>>string.punctuation
' ! " # $ % & \ ' ( ) * + , - . / : ; < = > ? @ [ \ ] ^ _ ` { | } ~ '
```

5.9 Formatting a String

Formatting a string is the process of infusing things in the string dynamically and presenting the string. Python supports three ways to format a given string, as follows:

- Using string formatting expression with `%` operator
- Using string formatting method `str.format()` and
- Formatting through **f-string** since ver. 3.6

5.9.1 String formatting using Formatting Expression (%)

Formatting a string, requires to inject a format specifier based on the datatype listed in Table 5. 3.

Table 5. 3 List of Format Specifiers used to format a string

Formatting Type	Format Specifiers	Description
Integer	%d	Print an Integer in decimal 10 base
Character	%c	Convert an integer to its corresponding character

Float-point	<code>%f</code>	Prints a floating-point number in standard decimal format
Exponential	<code>%e</code> or <code>%E</code>	Prints a floating-point number in scientific notation
String	<code>%s</code>	To print a string
Hexadecimal	<code>%x</code> or <code>%X</code>	Prints an integer in lowercase or uppercase hexadecimal format (base 16), respectively.
Octal	<code>%o</code> or <code>%O</code>	Prints an integer in lowercase or uppercase octal format (base 8), respectively.

While using `%` for string formatting is still supported in Python, it's considered an older approach. Here's how to use it with the `print` function:

```
print("Formatted string %s %d" % (value1, value2))
```

The above syntax can be interpreted as follows:

- The string you want to print is enclosed in double quotes.
- Within the string, placeholders for the values you want to insert are marked with `%s` for strings or `%d` for integers (and other format specifiers for different data types).
- After the closing quotation mark, you use the `%` operator again, followed by a parenthesis containing the values you want to insert into the placeholders.
- The order of values in the parenthesis must match the order of placeholders in the string.

```
>>> name = "Bob"
```

```
>>> age = 30
```

```
>>> print("Hello, my name is %s and I am %d years old." % (name, age))
```

```
Hello, my name is Bob and I am 30 years old.
```

Instead of using the placeholders within `print()`, Python also allows us to format a string first and print it next. See the example given below:

```
>>message = 'Hello, %s you are %d years old' % (name, age)
```

```
>>print(message)
```

```
Hello, Bob you are 30 years old
```

Use of parenthesis become compulsory, while passing multiple values to format a string. For single value it become optional.

```
>>> print('Hello, %s'% name)
```

```
Hello, Bob
```

Next example shows how to format a string including numbers in octal and hexadecimal format.

```
>>> a = 0xafe45
```

```
>>> b = 0o3456
```

```
>>> print('%d is equivalent to %x in hexadecimal' %(a, a))
```

```
720453 is equivalent to afe45 in hexadecimal
```

```
>>> print('%d is equivalent to %o in octal' %(b, b))
```

```
1838 is equivalent to 3456 in octal
```

String formatting with floating-point format-specifiers can be displayed in decimal and exponential form.

```
>>> a = 0.000023
>>> b = 111111111111111111.56
>>> 'a = %f, b = %f' % (a, b)
'a = 0.000023, b = 1111111111111111104.000000'
>>> 'a = %e, b = %e' % (a, b)
'a = 2.300000e-05, b = 1.111111e+17'
```

In the exponential form there will be a negative (-) or a positive (+) sign after **e**. A plus (+) sign indicates a large value and minus (-) means represents a very small value. Further we can decide in advance the width of the output, by specifying it with **%** sign. Field width is the width of the entire number and precision is the width towards the right. One can alter these widths based on the requirements. The default **precision** width is set to 6, for floating point numbers.

```
>>> a = 2.34
>>> print('a = %f' % a)
a = 2.340000
```

The default precision width for floating-point numbers using **%f** is indeed 6, as it shown in the output. Based on our requirement the width of the precision can be controlled, as follows:

```
>>> print('a = %.3f' % a)
a = 2.340
```

In the **print()** statement a value 3 is set, after the decimal point (**.**), that controls the width or length of the precision. That's why in the output **a = 2.340** is printed, instead of **a = 2.340000**. The width value should be placed in between **%** and **f**, and always be a positive integer. If the field width, is set to, more than the default or required width, than the data will be right aligns itself to adjust to the specified values.

```
>>> print('a = %f' % a)
a = 2.340000
>>> print('a = %5.3f' % a) #width 5 = as a's length and precision, (3 < 6)
a = 2.340
>>> print('a = %5.9f' % a) #width 5 = as a's length and precision, (9 > 6)
a = 2.340000000
>>> print('a = %6.3f' % a) #width 6 > a's length and precision, (3 < 6)
a = 2.340
>>> print('a = %10.9f' % a) #width 10 > a's length and precision, (9 > 6)
a = 2.340000000
```

Whenever we have used the precision value the output will be aligned either right or left. The first output, aligned to left, because we have not used field or precision width. But the second output, a value 5.3 is set, it means field width = 5 and precision = 3. So the value will be printed up to 3rd decimal place and the field width equal to the formatted numbers total length i.e. 5. That's why aligned to left.

In the 3rd `print()` statement, field width = 5 and precision = 9. Because the width is same as length then output will be left aligned but number of decimals placed is increased to 9 from 6. In the next line, field width = 6 and precision = 3 and the output is right aligned by single space after equal sign. Because, **field width - precision length (3) - length of decimal part (1) - 1 (decimal point) = 1**. In the last print, **field width (10) - precision (9) - length of decimal part (1) - 1 (decimal point) = -1**, output will be left aligned.

For proper alignment, a space can be left blank in the field width so that when a negative number is used, proper alignment is maintained.

```
>>> b = -2.340
>>> print('% .3f' %b)
-2.340
>>> print('%+.3f' %a)
+2.340
```

Like a minus (-) sign get printed for a negative value, similar to this we can also print the plus (+) sign for positive values, by inserting a plus (+) sign after the % sign.

Note: % formatting is not useful for larger string, because once you start using several parameters and longer strings, your code will quickly become much less easily readable and things are starting to look a little messy. To format long formatted string Python offers `str.format()` method.

5.9.2 String formatting using `str.format()` method

A string can be formatted using `str.format()` method and formatted string contain “replacement fields” surrounded by curly braces `{ }`. Anything that is not contained in braces is considered literal text, which is copied unchanged to the output. The syntax for a replacement field contains a field name, conversion character and a format specifier, as shown below:

```
replacement_field = "{"[field_name]!"[conversion]":["format_spec]}"
```

In less formal terms, the `replacement_field` can start with a `field_name`, that specifies the object whose value is to be formatted and inserted into the output of the replacement field. The `field_name` is optionally followed by a `conversion field`, which is preceded by an exclamation point `!`, and a `format_spec`, which is preceded by a colon `:`. These specify a non-default format for the replacement value.

The `field_name`, itself begins with an `arg_name`, that is either a number or a keyword. If it's a number, it refers to a positional argument, and if it's a keyword, it refers to a named keyword argument. Consider examples given below:

```
>>> '{lan} has ver {ver}'.format(lan = 'Python', ver = '3.x')
'Python has ver 3.x'
```

The string is formatted using **arg_name** only and inside the braces **lan** and **ver** is representing the argument name. Instead of **arg_name**, position of the arguments can be used by specifying their position inside the curly braces (**{}**) and by default an empty pair of braces **{}**, represents positional only.

```
>>> '{} has ver {}'.format('Python', '3.x')
'Python has ver 3.x'
>>> '{1} has ver {0}'.format('3.x', 'Python')
'Python has ver 3.x'
```

Position index starts from 0 and using argument positions can be passed in any order. If the numerical **arg_name** in a format string are **0**, **1**, **2** in a sequence, they can all be omitted (not just some) and the argument numbers will be automatically inserted in that order. Mixing of default and manual numbering will raised a **ValueError**.

```
>>> '{} has ver. {1}' .format('Python', '3.x')
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: cannot switch from automatic field numbering to manual field specification
```

Further dictionary keys can be used as the position of the argument for formatting a string. See the example given below:

```
>>> language = {'lan': 'Python', 'ver': '3.x'}
>>> '{lan} has ver. {ver}' .format(**language)
'Python has ver. 3.x'
```

The conversion field causes the type correction before formatting the given string. Formatting of any value is done by the **__format__()** method of that value itself. However, in some cases, it is desirable to force a type to be formatted as a string, by overriding its own definition of formatting. This ensures the value is treated as a plain string and formatted according to the format specifiers you provide in the string (like **.2f** for two decimal places). Details of all conversion flag that can be used to convert the string to a specific type is listed in Table 5.4.

Table 5.4 List of flags used in conversion field

Conversion Flags	Description
!s	Calls the str() method
!r	Calls the repr() function, returns a printable character
!a	Calls the ascii() function, return a printable representation of an object. Ignores the non-ascii characters using \x , \u , \U

Using these flags, we can check further a string is in printable form or not. See the coding example listed below:

```
#Converts the string in printable form
>>'{s!r}'.format(s = '¡¢£ Pyðþðþthon')
"¡¢£ Pyðþðþthon"
#Converts all non-ascii characters to ascii code
>>'{s!a}'.format(s = '¡¢£ Pyðþðþthon')
"'\xa1\xa2\xa3 Py\xbc\xbd\xbethon'"
#Strings of non-ascii characters
>>' \xa1\xa2\xa3'
'¡¢£'
```

The last **format_spec**, in **replacement_field**, contains a specification of how the value should be presented, including such details as field width, alignment, padding, decimal precision and so on. Each value type can define its own “formatting mini-language” or interpretation of the **format_spec**.

A **format_spec** field can also include nested replacement fields within it. These nested replacement fields may contain a field name, conversion flag, and format specification, but deeper nesting is not allowed. The replacement fields within **format_spec** are substituted before the **format_spec** string is interpreted. This allows the formatting of a value to be dynamically specified. A general form of format specifier is given below and in Table 5.5, listed every part of a standard format specifier with description.

```
format_spec = [[fill] align] [sign] [z] [#][0][width] [grouping_option]
[.precision][type]
```

Table 5.5 Various types of Field can be used in format specifier

Field Value	Description
fill	Filled the string with any character when passed with valid align value
align	Explained in Table
sign	‘+’ or ‘-’ or space with numeric values only
width	A positive integer, controlling the length of the formatted string
grouping action	‘-’ or ‘,’
precision	A decimal integer, indicating number of digits after the decimal point
type	Determines how the data should be represented

With an align value any filled character can be preceded that can be any character and defaults to a space if omitted. It is not possible to use a literal curly brace (“{” or “}”) as the fill character in a formatted string literal or when using the **str.format()** method. Various alignment options are listed in Table 5.6:

Table 5.6 Details of alignment options with meaning

Alignment Options	Description
‘<’	Forces the field to be left-aligned within the available space (this is the default for most objects).

'>'	Forces the field to be right-aligned within the available space (this is the default for numbers).
'='	Forces the padding to be placed after the sign (if any) but before the digits. This is used for printing fields in the form '+000000120'. This alignment option is only valid for numeric types. It becomes the default for numbers when '0' immediately precedes the field width.
'^'	Forces the field to be centered within the available space.

Some examples of string formatting based on **align** value with predefined **fill** character and width value (the length of the formatted string) is illustrated next, using keyword argument:

```
#Text left aligned with fill character and specified width with
>>'{text:{fill}{align}20}'.format(fill='#',text = 'AB',align='<')
'AB#####'
#Text centre aligned with fill character and specified width with
>>'{text:{fill}{align}20}'.format(fill='#',text = 'AB',align='^')
'#####AB#####'
```

Note: unless a minimum field width is defined, the field width will always be the same size as the data to fill it, so that the alignment option has no meaning in this case.

```
#width is less than the length of the text
>>'{text:{fill}{align}}'.format(fill='#',text = 'AB', align='^')
'AB'
>>'{text:{fill}{align}3}'.format(fill='#',text = 'AB',align='^')
'AB#'
>>'{text:{fill}{align}4}'.format(fill='#',text = 'AB',align='^')
'#AB#'
```

Formatting any string of numbers with a positive or negative sign can be done by passing the **sign** value just before the **width** of the string. All the possible signs with their details are listed in Table 5.7.

Table 5.7 Details of sign fields

Option	Meaning
'+'	indicates that a sign should be used for both positive as well as negative numbers.
'-'	indicates that a sign should be used only for negative numbers (this is the default behavior).
Space	indicates that a leading space should be used on positive numbers, and a minus sign on negative numbers.

```
>>>'{:+} {: -} {: }'.format(3.56,-2.52,-56)
'+3.56 -2.52 -56'
```

```
>>>'{: -} {: } {: -}'.format(3.56, -2.52, -56)
'3.56 -2.52 -56'
```

The optional literal **z** forces negative zero floating-point values to positive zero after rounding to the format precision. This formatting option is added in version 3.11. In many fields of mathematics negative zero is surprising or unwanted minus (**-**), especially in the context of displaying an (often rounded) numerical result. Using this optional **z**, for floating-point presentation types (f, g, etc., as defined by the format specification documentation), normalized to a negative zero to positive zero.

```
#with z
>>'{:z.2f}'.format(-0.0001)
'0.00'

#without z
>>'{: .2f}'.format(-0.0001)
'-0.00'
```

The **#** option causes the “alternate form” to be used for the conversion. The alternate form is defined differently for different types. This option is only valid for integer, float and complex types. For integers, when binary, octal, or hexadecimal format is used, this option adds the respective prefix **'0b'**, **'0o'**, **'0x'**, or **'0X'** to the output value. See the examples given below:

```
#with # before a type
>>'20 in binary {:#b} {:b}'.format(20, 20)
'20 in binary 0b10100 10100'

#without #
>>'20 in binary {:b}'.format(20)
'20 in binary 10100'
```

In this above illustration a type **'b'** is used for binary number and using **#** just before it prefixes **'0b'** before the number. If the type is not preceded by a **'#'**, then simply returns the conversion without any prefix. The comma (,) option signals the use of a comma for a thousand separator.

```
#with width and thound separator
>>'{: {fill}{align}12,}'.format(1234.8, fill='#', align='<')
'1,234.8#####'

#Thousand separator in integer
>>'{: ,}'.format(1023456)
'1,023,456'
```

The underscore option signals the use as a placeholder to improve the readability for large numbers. It is used as a thousand separator for integer (decimal type) and floating point literals. For other integer presentation types **'o'**, **'x'**, and **'X'**, underscores will be inserted after every 4 digits.

```
>>> '{:_b}'.format(1023456)
'1111_1001_1101_1110_0000'
>>> '{:_d}'.format(1023456)
'1_023_456'
>>> '{:_o}'.format(1023456)
'371_6740'
>>> '{:_X}'.format(1023456)
'F_9DE0'
```

Use of **precision** value with 'f' format specifier for fixed-point notation for displaying a specific number of decimal places. While 'g' format specifier for general notation (choosing between fixed-point or scientific notation for best readability), specifies number of significant digits (leading zeros don't count). In this case, it chooses fixed-point notation rounding to 3 significant digits.

```
>>> '{:.3f}'.format(2.456890000)
'2.457'
>>> '{:.3g}'.format(2.456890000)
'2.46'
```

For string **precision** field indicates the maximum field size - in other words, how many characters will be used from the field content. The **precision** is not allowed for integer presentation types.

```
>>> '{:.3s}'.format('Python') # Taking 3 characters
'Pyt'
>>> '{:0s}'.format('Python') # 0-length string
''
```

Finally, the **type** determines how the data should be presented. The available presentation types for string and integer type are listed in Table 5.8.

Table 5.8 List of available string and integer presentation type

Type	Meaning
's'	String format. This is the default type for strings and may be omitted.
B	Binary format. Outputs the number in base 2.
C	Character. Converts the integer to the corresponding unicode character before printing.
D	Decimal Integer. Outputs the number in base 10.
'o'	Octal format. Outputs the number in base 8.
'x'	Hex format. Outputs the number in base 16, using lower-case letter.
'X'	Hex format. Outputs the number in base 16, using upper-case letters.
N	Number. This is the same as 'd', except that it uses the current locale setting (defined within current system settings) to insert the appropriate number separator characters.

```
>>> "int: {0:d}; hex: {0:x}; oct: {0:o}; bin: {0:b}".format(42)
'int: 42; hex: 2a; oct: 52; bin: 101010'
```

In addition to the above presentation types, integers can be formatted with the floating-point presentation types listed in (except 'n' and **None**) Table 5.9. When doing so, **float()** is used to convert the integer to a floating-point number before formatting.

Table 5.9 Available presentation types for float and Decimal values

Type	Meaning
'e'	Floating point
'E'	Scientific notation. Same as 'e' except it uses an upper case 'E' as the separator character.
'f'	Fixed-point notation
'F'	Fixed-point notation. Same as 'f', but converts nan to NAN and inf to INF.
'g'	General format
'G'	General format. Same as 'g' except switches to 'E' if the number gets too large. The representations of infinity and NaN are uppercased, too.
'n'	Number. This is the same as 'g', except that it uses the current locale setting to insert the appropriate number separator characters.
'%'	Percentage. Multiplies the number by 100 and displays in fixed ('f') format, followed by a percent sign.
None	For float this is the same as 'g', except that when fixed-point notation is used to format the result, it always includes at least one digit past the decimal point.

In addition to the string's **format()** method, a single object may also be formatted, using the syntax **format(object, formatspec)**, built-in function (which the method uses internally). The format specifications listed in Table 5.3 need to pass in place of **formatspec** inside **format()**, to get the formatted string. Next some of the examples are illustrated to help you understand the working of **format()** function.

```
>>format(13, 'o')
'15'
>>format(2345.6, '.2f')
'2345.60'
>>format(10000000, ',')
'10,000,000'
>>format(75, '#b')
'0b1001011'
>>format('wer', '>20')
'                wer'
>>format('wer', '#>20')
'#####wer'
>>format(-0., 'z')
'0.0'
>>format(12, '%')
'1200.000000%'
```

```
>>format(23.687,'e')
'2.368700e+01'
>>format(23.687,'g')
'23.687'
```

5.9.3 String formatting using f-string

Strings in Python are usually enclosed within double quotes (“”) or single quotes (‘’). To create **f-strings**, you only need to add character ‘**f**’ or ‘**F**’ before the opening quotes of the string. For example, “**This**” is a string, whereas **f“This**” is an **f-String**.

When using **f-strings** to display strings, you only need to specify the names of the variables inside a placeholder (**{}**). During the execution, all the names are replaced with their respective values. If you have multiple variables in the string, you need to enclose each of the variable names inside the placeholder. Suppose you have two variables namely **language**, and **school**, and a string can be formed as follows:

```
>>language = 'Python'
>>school = 'Computer Science & Engineering'
>>message = f'We are learning {language} from {school}'
>>message
'We are learning Python from Computer Science & Engineering'
```

Notice how the variables **language** and **school** are replaced with their values. As **f-Strings** are evaluated at runtime, you can evaluate valid Python expressions on the fly. In the next illustration, **num1** and **num2** are two variables. To calculate their product, you may insert the expression **num1 * num2** inside a placeholder, and the expression will be evaluated, as follows:

```
>>num1, num2 = 3, 4
>>print(f'The product of {num1} and {num2} is {num1 * num2}')
The product of 3 and 4 is 12
```

In an **f-string**, **{var_name}**, **{expression}** serve as placeholders for **variables** and **expressions**, and are replaced with the corresponding values at runtime. Any string can be formatted using multiple **f-strings**, remember that you need to place an **f** in front of each line of a multiline string.

```
>>name = "Eric"
>>profession = "comedian"
>>affiliation = "Monty Python"
>>message = (f'Hi {name}. 'f'You are a {profession}. 'f'You were in
{affiliation}.")
>>message
'Hi Eric. You are a comedian. You were in Monty Python.'
```

If we don’t put an **f** in front of each individual string, then placeholder **{}**, is considered as part of the string. A string can also spread over multiple lines, by escaping a return with a backslash ‘\’:

```
>>msg = f'Hello {name} '\
```

```
f'you are a {profession} '\
f'you were in {affiliation}'
>>msg
'Hello Eric you are a comedian you were in Monty Python'
```

The **'f'**, in f-strings may as well stand for “fast.”, f-strings are faster than both **%**-formatting, and **str.format()**. Let's see how to take care quotation mark within an f-string. While working with quotations just make sure that, similar type of quotation mark on the outside of the f-string as you are using in the expression, as follows:

```
>>f"{'Hello Python'}"
'Hello Python'
>>f'{"Hello Python"}'
'Hello Python'
>>f'''{"Hello Python"}'''
'Hello Python'
```

While formatting a dictionary within a string using f-string the double quotes needs to be surrounded and keys must be retrieve using single quote. Using of single quote will leads to a **SyntaxError**, as follows:

```
>>language = {'name': 'Python', 'ver': 3.11}
>>f'{language['name']} has verion {language['ver']}'
SyntaxError: f-string: unmatched '['
```

The correct f-string will be:

```
>>language = {'name': 'Python', 'ver': 3.11}
>>f'{language['name']} has verion {language['ver']}'
'Python has verion 3.11'
```

There is a small difference between **str.format()** and f-strings. i.e., the difference is in how index lookups are performed. In **str.format()**, index values that do not look like numbers are converted to strings:

```
>>d = {'a': 10, 'b': 20}
>>'a={d[a]}'.format(d=d)
'a=10'
```

In **.format()** method, you need to use keyword arguments that match the placeholder names in the string. Non-numeric keys (like variable names) are treated as literal strings within the curly braces. The f-strings enable direct attribute access using string keys within curly braces **{}**. This means you can use a variable name that corresponds to an object's attribute.

```
>>f'a={d["a"]}'
'a=10'
```

This difference is required because otherwise you would not be able to use variables as index values:

```
>>a = 'b'
#using variable as key
>>f'a={d[a]}'
'a=20'
```

The `.format()` method allows you to insert values into strings using curly braces `{}`. However, it has limitations on the types of expressions you can use within those braces. It's generally recommended to use valid Python expressions within `.format()` to avoid unexpected behavior. If you need more flexibility in formatting expressions, consider using f-strings.

Note: If you need more flexibility in formatting expressions, consider using f-strings n deep dive further to PEP 498²², which covers other details about the `f`-string and `str.format()`.

Key Takeaway's

- Strings in Python are immutable, meaning once created, they cannot be changed.
- Use the `+` operator to concatenate strings.
- Access characters in a string using indexing with square brackets `[]`. Indexing starts at 0.
- Retrieve parts of strings using slicing with the syntax `[start : stop : step]`.
- During slicing, if `start` is omitted, it defaults to 0. If `stop` is omitted, it defaults to the length of the string. If `step` is omitted, it defaults to 1.
- Negative values can be used for start, stop, and step. A negative index counts from the end of the string.
- Slicing will not throw an error for indexes out of range; it will simply return an empty string or the available characters.
- Use backslashes `\` to insert special characters into strings, like `\n` for a newline or `\\` for a backslash.
- Python has many built-in string methods for common tasks like `.upper()`, `.lower()`, `.strip()`, `.split()`, and `.replace()`.
- Use `.format()` or `f`-strings (formatted string literals) to embed expressions inside string literals.
- Prefix strings with `r` to treat backslashes as literal characters and not escape characters.
- Use triple quotes `"""` or `'''` for strings that span multiple lines.
- Python strings are Unicode by default, which means they can include characters from most of the world's writing systems.
- To reverse a string use a negative step to reverse a string, like `[::-1]`

*****END OF CHAPTER*****

²² <https://peps.python.org/pep-0498/#format-specifiers>